

Hugging Face

Models, Datasets, Spaces, Transformers, PEFT, Inference — the complete ecosystem

01 What is Hugging Face?

Hugging Face is the central hub of the open-source AI ecosystem. It hosts 800,000+ models, 150,000+ datasets, and provides the **transformers** library — the standard way to use pre-trained models in Python. Think of it as GitHub for AI models.

Component	What It Is	Use It For
Hub (huggingface.co)	Model and dataset repository	Find, download, and share models and datasets
transformers library	Python library for using pre-trained models	Load and run any model — NLP, vision, audio
datasets library	Fast dataset loading and processing	Load benchmarks and training datasets efficiently
Spaces	Hosted Gradio/Streamlit apps	Demo models and apps for free
Inference API	Hosted model inference	Call models via HTTP without running them locally
PEFT / TRL	Fine-tuning libraries	LoRA, QLoRA fine-tuning on consumer hardware
Accelerate	Multi-GPU/TPU training	Scale training across devices easily
Gradio	UI library for ML demos	Build interactive model demos in Python

02 The transformers Library

The **transformers** library is the core of HuggingFace. It provides a unified API for loading and running thousands of pre-trained models with just 2-3 lines of code.

Installation

```
pip install transformers torch
pip install transformers[sentencepiece] # for multilingual models
pip install accelerate # for faster inference and training
```

Pipeline — Simplest Way to Use Models

The **pipeline** function wraps model + tokenizer + post-processing into one simple call.

```

# Common pipeline tasks
from transformers import pipeline

# Text generation (LLM)
generator = pipeline("text-generation", model="Qwen/Qwen2.5-Coder-7B")
result = generator("def fibonacci(n):", max_new_tokens=100)

# Question answering (extractive)
qa = pipeline("question-answering",
               model="deepset/roberta-base-squad2")
result = qa(question="What is RAG?",
             context="RAG is Retrieval-Augmented Generation...")

# Text classification (sentiment)
classifier = pipeline("sentiment-analysis")
result = classifier("I love this product!")
# [{"label": "POSITIVE", "score": 0.9998}]

# Named Entity Recognition
ner = pipeline("ner", aggregation_strategy="simple")
result = ner("Ramesh works at Infosys in Hyderabad.")

# Summarization
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
result = summarizer(long_text, max_length=130, min_length=30)

# Zero-shot classification (no training needed)
classifier = pipeline("zero-shot-classification")
result = classifier("This is about sports",
                    candidate_labels=["sports", "finance", "technology"])

```

AutoModel — More Control

```

# Load any model manually for more control
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_name = "meta-llama/Llama-3.2-3B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype=torch.float16, device_map="auto")

inputs = tokenizer("Hello, explain RAG in one sentence:", return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

03 Sentence Transformers — Embeddings for RAG

The **sentence-transformers** library (built on HuggingFace) is the go-to for generating text embeddings. Essential for RAG pipelines.

```
# Embeddings with sentence-transformers
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

# Embed documents
docs = ["Paris is the capital of France",
        "Berlin is the capital of Germany",
        "Machine learning is a type of AI"]
doc_embeddings = model.encode(docs)

# Embed query and find most similar
query = "What is the capital of France?"
query_embedding = model.encode(query)

# Cosine similarity
scores = util.cos_sim(query_embedding, doc_embeddings)
# scores = [[0.91, 0.12, 0.08]] – first doc most similar

# Cross-encoder for reranking (more accurate)
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
pairs = [(query, doc) for doc in docs]
rerank_scores = reranker.predict(pairs)
```

04 Fine-Tuning with PEFT and LoRA

PEFT (Parameter-Efficient Fine-Tuning) lets you fine-tune large models by training only a small set of adapter weights (1-3% of total parameters), dramatically reducing GPU memory and training time.

LoRA (Low-Rank Adaptation) is the most popular PEFT method. Instead of updating all model weights, it adds small trainable matrices (rank decomposition) at specific layers.

```
# QLoRA fine-tuning (4-bit quantization + LoRA)
from transformers import AutoModelForCausalLM, TrainingArguments
from peft import LoraConfig, get_peft_model
from trl import SFTTrainer
import torch

# Load base model in 4-bit
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-3B",
    load_in_4bit=True, # QLoRA: 4-bit quantization
    device_map="auto")

# LoRA config
lora_config = LoraConfig(
    r=16, # rank (smaller = fewer params)
```

```

lora_alpha=32,          # scaling factor
target_modules=["q_proj", "v_proj"], # which layers to adapt
lora_dropout=0.05,
task_type="CAUSAL_LM")

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 3,407,872 || all params: 3,215,843,328 || 0.11%

# Train with SFT Trainer
trainer = SFTTrainer(
    model=model, train_dataset=dataset,
    args=TrainingArguments(output_dir="./output", num_train_epochs=3))
trainer.train()

```

05 Inference API and Spaces

Inference API — Call Models Without Running Them

```

# Call any HuggingFace model via API
import requests

API_URL = "https://api-inference.huggingface.co/models/gpt2"
headers = {"Authorization": "Bearer hf_YOUR_TOKEN"}

response = requests.post(API_URL,
    headers=headers,
    json={"inputs": "The future of AI is"})

print(response.json())

```

Gradio — Build Model Demos in Minutes

```

# Simple Gradio RAG demo
import gradio as gr

def answer_question(question, context):
    # your RAG logic here
    return rag_chain.invoke({"query": question})["result"]

demo = gr.Interface(
    fn=answer_question,
    inputs=[gr.Textbox(label="Question"),
            gr.Textbox(label="Context")],
    outputs=gr.Textbox(label="Answer"),
    title="My RAG Demo")

demo.launch() # Opens at http://localhost:7860
# demo.launch(share=True) # Public URL for sharing

```

Library	Install	Key Use
transformers	<code>pip install transformers</code>	Load and run any pre-trained model
sentence-transformers	<code>pip install sentence-transformers</code>	Text embeddings for RAG and semantic search
datasets	<code>pip install datasets</code>	Load training/benchmark datasets
peft	<code>pip install peft</code>	LoRA, QLoRA, and other efficient fine-tuning methods
trl	<code>pip install trl</code>	SFT, RLHF, DPO training loops
accelerate	<code>pip install accelerate</code>	Multi-GPU training, mixed precision
bitsandbytes	<code>pip install bitsandbytes</code>	4-bit/8-bit quantization for loading large models
gradio	<code>pip install gradio</code>	Build and share model demo UIs
evaluate	<code>pip install evaluate</code>	Standard metrics: ROUGE, BERTScore, accuracy, F1
tokenizers	<code>pip install tokenizers</code>	Fast tokenization library (Rust-backed)
optimum	<code>pip install optimum</code>	Optimize models for inference (ONNX, OpenVINO)
diffusers	<code>pip install diffusers</code>	Stable Diffusion and other image generation models